

Machine Learning in Practice I

Lecture 1

Antal Jakovac, 2025



About the course

- Course Title: "Machine Learning in Practice I"
- instructor: antal.jakovac@uni-corvinus.hu
- course objectives
 - ➔ why and when to use Machine Learning (ML)
 - ➔ data handling methods
 - ➔ core ML models before deep learning
 - ➔ how to build an ML project



- about the course
 - ➔ evaluation: written exams (together with exercises)
 - ➔ learning material: course slides, books, podcasts, etc.
 - ➔ books:
 - An Introduction to Statistical Learning
(G. James , D. Witten , T. Hastie , R. Tibshirani, <https://www.statlearning.com/>)
 - Designing machine learning systems (Chip Huyen)
 - ➔ github repository: <https://github.com/ajakovac/ML-in-Practice-I>



- about the course

- ➔ evaluation: written exams (together with exercises)
- ➔ learning material: course slides, books, podcasts, etc.

- ➔ books:

- An Introduction to Statistical Learning
(G. James , D. Witten , T. Hastie , R. Tibshirani, <https://www.statlearning.com/>)

- Design *#git clone project from command line*

- ➔ github repository

```
git clone https://github.com/ajakovac/ML-in-Practice-I.git  
cd ML-in-Practice-I
```

#read the README.md file and follow instructions

Introduction



- why do we need Artificial Intelligence (Machine Learning)?

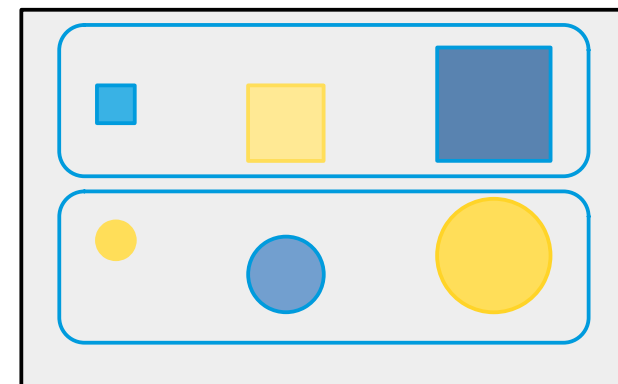
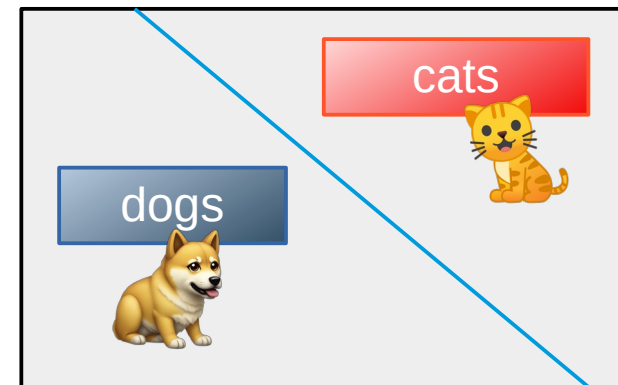
	observation	modelling	computation	conclude/action
historic times	human	human	human	human
machines	machine	human	human	human
computers	machine	human	machine	human
AI	machine	machine	machine	human

- task of AI is not to solve an actual problem, but solve modelling
- we do not know the model it uses → it can be smarter than us!



- Traditional Programming vs. ML
 - ➔ traditional: data + rules → output
 - ➔ ML: data + output → rules
- when to use ML?
 - ➔ Automation of complex tasks → heuristics
 - ➔ Adaptability
 - ➔ Performance on large data
 - ➔ Real-world examples (email spam, face detection, recommendation systems)

- Types of ML; traditionally
 - supervised:
 - show data + labels → learns the assignment
 - examples: classification, regression
 - unsupervised:
 - find patterns in unlabeled data
 - not unique → context, magnitude
 - examples: clustering, dimensionality reduction
 - reinforcement:
 - learn by interacting with the environment
 - not covered in this course





- Types of ML; other points of view
 - task-driven (System I)
 - define a task (classification, prediction, control)
 - show examples of good (eventually bad) solutions
 - train models to optimize for success on that task
 - includes most supervised/reinforcement learning examples
 - data-driven (System II)
 - reveal structures by showing similar data
 - does not need explicit labelling
 - LLM's, autoencoders, similarity learning
 - recently popular (self-supervised learning, foundation models, and retrieval-based systems)



- Typical ML tasks
 - classification
 - face, dog breeds, bird songs classification
 - spam detection, disease diagnosis, sentiment analysis
 - lot of labeled data
 - regression (price prediction, weather forecasting, energy demand)
 - clustering (customer segmentation, document grouping, biological cell types)
 - compression (PCA, visualization, autoencoders, embeddings)



- Typical ML tasks
 - ➔ decision making (robotics, game playing, resource allocation, monitoring)
 - ➔ generation (text, translation, image, data, molecular structure, code)
 - ➔ outlier analysis (email spam, suspicious customer, fraud detection, etc.)
 - ➔ recommendation systems (movies, jobs, code completion, etc.)
 - ➔ ranking/retrieval (search engines, document/data retrieval)



- somewhat overlapping areas; e.g. chemical plant monitoring system
 - ➔ anomaly detection
 - ➔ classification
 - ➔ time-series forecasting
 - ➔ reinforcement learning (control), recommendation systems



- ML algorithms (pre-deep neural networks)
 - Linear regression, logistic probability
 - Bayesian analysis, probability theory
 - Support Vector Machines
 - Decision Trees, ensemble methods
 - k-nearest neighbour method
 - Principal Component Analysis
 - Extreme Learning Machine
- majority of ML applications in production are not DNN!



Tools

- language: Python
- packages:
 - numpy: numerical computing
 - scipy, scikit-learn: classical ML tools
 - pandas: data manipulation
 - matplotlib, seaborn: visualization



Tools

- Python virtual environment:

```
> python -m venv .venv
```

- activate environment:

- Linux/Windows WSL:

```
> source .venv/bin/activate
```

- Windows PowerShell:

```
> .venv\Scripts\Activate
```

- Install necessary packages within the environment

```
> pip install matplotlib numpy scipy scikit-learn pandas seaborn ipykernel
```

- automation:

```
> pip freeze > requirements.txt
```

```
> pip install -r requirements.txt
```



Tools for code development

- notebook services: Jupyter, Google Colab
- advanced editors: Visual Studio Code or similar
- Linux-like environment in Windows: WSL
- docker: containerize your work – like a standalone machine
- communication: API endpoints (Python fastAPI or Flask)

Typical ML workflow



- Typical ML workflow
 - 1) Problem definition
 - 2) Data collection
 - 3) Data cleaning & preprocessing
 - 4) Feature engineering
 - 5) Model selection
 - 6) Training & validation
 - 7) Evaluation
 - 8) Deployment & monitoring



- Typical ML workflow

- 1) Problem definition
- 2) Data collection
- 3) Data cleaning & preprocessing
- 4) Feature engineering
- 5) Model selection
- 6) Training & validation
- 7) Evaluation
- 8) Deployment & monitoring



- what is the problem we want to solve? (classification, regression, clustering, ...)
- is ML an adequate tool?
- what does success look like? (accuracy? low error? business impact?)
- what are the domain constraints and goals (e.g. medicine data availability, privacy → practical, ethical or technical limits)
- example: predict customer churn or classify handwritten digits



- Typical ML workflow

- 1) Problem definition
- 2) Data collection 
- 3) Data cleaning & preprocessing
- 4) Feature engineering
- 5) Model selection
- 6) Training & validation
- 7) Evaluation
- 8) Deployment & monitoring

- enough data
- sampling methods → diverse data
- labelled data (if supervised)
- example: downloading a CSV of housing prices or collecting user logs



- Typical ML workflow

- 1) Problem definition
- 2) Data collection
- 3) Data cleaning & preprocessing
- 4) Feature engineering
- 5) Model selection
- 6) Training & validation
- 7) Evaluation
- 8) Deployment & monitoring

- 
- missing values, duplicates, outliers
 - normalize, scale, encode categorical features
 - convert raw inputs into usable formats
 - example: fill missing ages with median; scale prices to [0,1]



- Typical ML workflow

- 1) Problem definition
- 2) Data collection
- 3) Data cleaning & preprocessing
- 4) Feature engineering
- 5) Model selection
- 6) Training & validation
- 7) Evaluation
- 8) Deployment & monitoring

- 
- create meaningful input features
 - transform or combine raw attributes
 - select or construct new features based on domain knowledge
 - example: from date of birth → compute age; combine “city” and “job type”



- Typical ML workflow

- 1) Problem definition
- 2) Data collection
- 3) Data cleaning & preprocessing
- 4) Feature engineering
- 5) Model selection
- 6) Training & validation
- 7) Evaluation
- 8) Deployment & monitoring

- choose a suitable algorithm (e.g. linear regression; SVM; RF; k-NN)
- consider interpretability, complexity, training time
- example: try logistic regression first for classification, then maybe a random forest, or a deep neural network



- Typical ML workflow

- 1) Problem definition
- 2) Data collection
- 3) Data cleaning & preprocessing
- 4) Feature engineering
- 5) Model selection
- 6) Training & validation
- 7) Evaluation
- 8) Deployment & monitoring

- 
- split data into training and validation sets (and possibly test)
 - train on one part, validate on another
 - tune hyperparameters (e.g., via cross-validation)
 - example: train a model on 80% of the data, validate on 20%



- Typical ML workflow
 - 1) Problem definition
 - 2) Data collection
 - 3) Data cleaning & preprocessing
 - 4) Feature engineering
 - 5) Model selection
 - 6) Training & validation
 - 7) Evaluation
 - 8) Deployment & monitoring

Use metrics appropriate for the task:

- classification: accuracy, precision, recall, F1
- regression: MSE, MAE, R^2
- plot confusion matrices, ROC curves, residuals
- example: check precision/recall on spam classification



- Typical ML workflow

- 1) Problem definition
- 2) Data collection
- 3) Data cleaning & preprocessing
- 4) Feature engineering
- 5) Model selection
- 6) Training & validation
- 7) Evaluation
- 8) Deployment & monitoring

- integrate the model into production (web app, API, etc.)
- monitor for drift, performance degradation
- retrain if needed (based on new data or feedback)
- example: host model on a server and track prediction accuracy over time

END OF LECTURE 1